
UTM

UNIVERSITI TEKNOLOGI MALAYSIA

**FACULTY OF COMPUTING**

**SEMESTER 2 2025/2026**

---

**SECP3133 – HIGH PERFORMANCE DATA PROCESSING**

**SECTION 01**

# Project 1: Optimising High-Performance Data Processing for Large-Scale Web Crawlers

**LECTURER:**

**PROF. MADYA. TS. DR. MOHD SHAHIZAN BIN OTHMAN**

**SUBMISSION DATE: 24 MAY 2026**

**GROUP: CODE COOKIES**


| STUDENT NAME | MATRIC NO |
|---|---|
| NAJMA SHAKIRAH BINTI SHAHRULZAMAN | A23CS0140 |
| NURUL ASYIKIN BINTI KHAIRUL ANUAR | A23CS0162 |
| HARINI A/P SANGARAN | A23CS0081 |

# Table of Contents

## 1.0 Introduction

This section introduces the project background, objectives, and target website of the project, providing the foundation for understanding the design and implementation of the high-performance data processing pipeline developed for large-scale job listing data collected from JobStreet.

## 1.1 Project Background

The increase of digital platforms has transformed the job market, making online employment marketplace a critical source of real-time employment intelligence. Platforms such as JobStreet aggregate millions of job listings across Southeast Asia, which includes diverse industries, employment categories, and salary structures. The large-scale data introduces both opportunities and challenges, while the volume of information is substantial, extracting, cleaning and processing it at scale demands a systematic and computationally efficient pipeline.

This project explores how to address that challenge using high-performance data processing methods. The pipeline is applied to the JobStreet website, a leading online employment marketplace and job search platform in Asia, primarily operating in Malaysia, Singapore, the Philippines, Indonesia, and Thailand.

## 1.2 Objectives

The primary objectives of this project are as follows:

1. To implement a high-throughput web crawler capable of collecting at least 100,000 job listings records from JobStreet.
2. To apply data cleaning methods to ensure the quality, consistency, analytical readiness of collected dataset.
3. To apply two high-performance computing optimization techniques and identify their impact on the pipeline performance.
4. To evaluate and compare pipeline performance using metrics including execution time, CPU utilization, peak memory consumption, and throughput.

### 1.3 Target Website and Data to Extract

The data source and target website for this project is JobStreet (my.jobstreet.com) which is one of the largest online employment platforms in Southeast Asia, operating across Malaysia, Singapore, Indonesia, Philippines and Thailand. The platform publishes structured job listing data that includes attributes such as job title, company name, location, salary range, posting date, employment type, and industry classification. Figure 1.1 shows the layout of the JobStreet website.

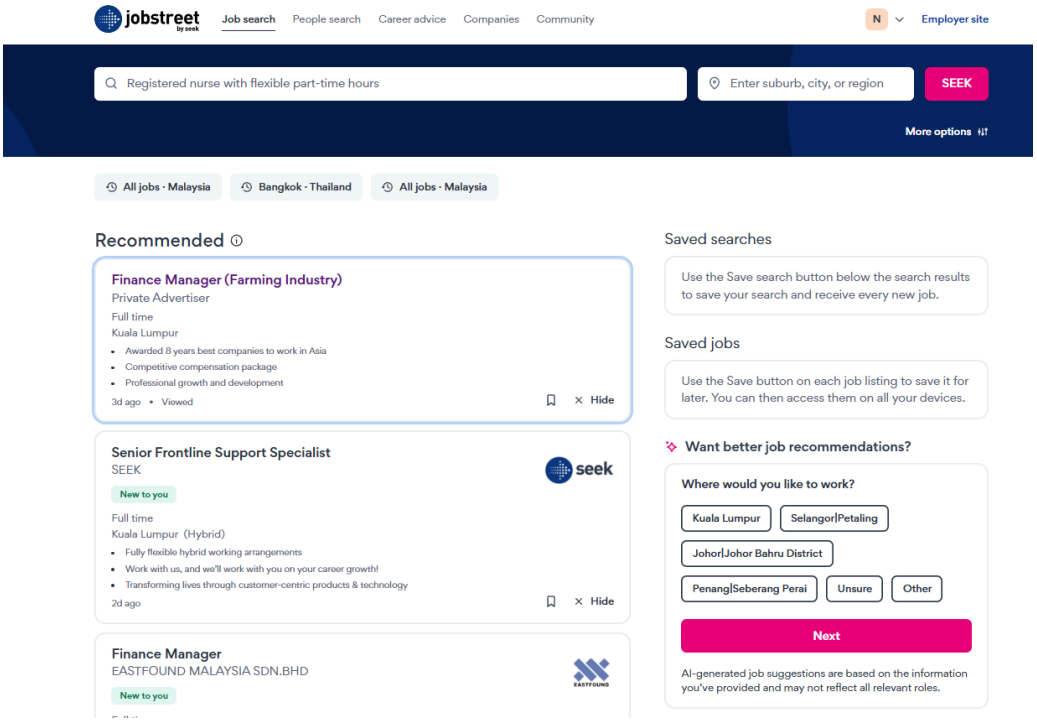



***Figure 1.1:*** *JobStreet Website*

## 2.0 System Design and Architecture

This chapter discusses the overall system architecture and workflow developed for the JobStreet web scraping project. It explains the system layers, tools, and frameworks used for web crawling, data processing, optimisation, and data analysis. In addition, the chapter outlines the responsibilities of each team member throughout the project development process.

## 2.1 System Architecture

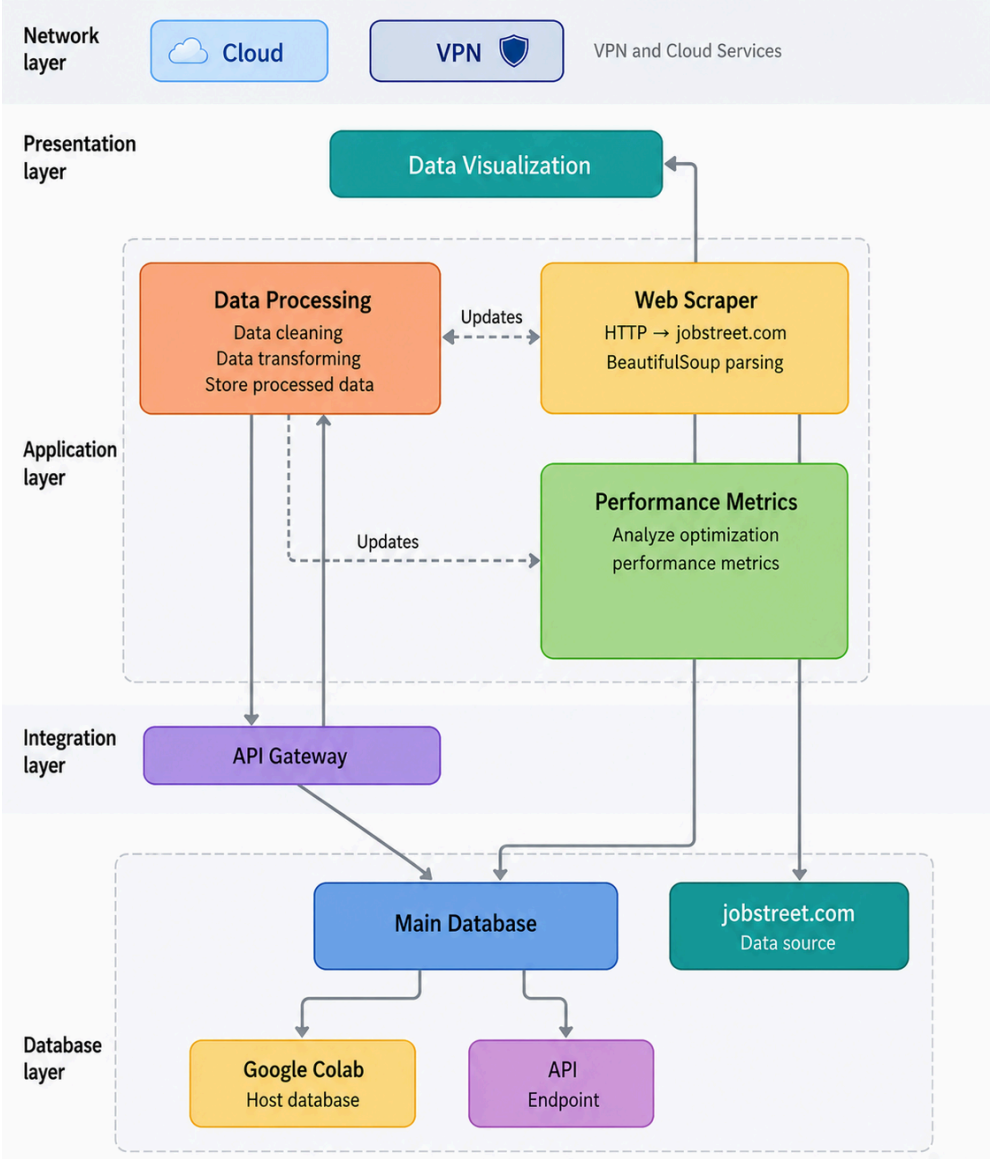



***Figure 1 :*** *System Architecture of Web Crawler*

### 1. Network Layer

The network layer provides secure and reliable communication between the web scraping system and external services. This layer uses cloud services and VPN technology to ensure secure data transmission during scraping operations. Since large-scale web scraping involves handling traffic and frequent HTTP requests, the network layer maintains high throughput and ensures uninterrupted communication with JobStreet.com. This enables the system to collect and process data efficiently in real time.

**2. Presentation Layer**

The presentation layer displays the analysed and optimised job market data in a clear and user-friendly format. This layer uses data visualisation techniques such as charts and dashboards to present performance metrics and scraping results which allows users to compare processing time, memory usage, CPU usage and throughput generated from the analysis modules.

**2. Application Layer**

a) **Web Scraper Module**

The web scraper module is responsible for sending HTTP requests to JobStreet.com using the Requests library and extracting relevant job-related information through BeautifulSoup parsing. The module collects data such as countries, job titles, company names, locations, salary, job industry, employment mode and when the jobs were posted in jobstreet. This process enables automated large-scale data collection from the website efficiently.

b) **Data Processing Module**

The data processing module is responsible for cleaning, validating and transforming the scraped data. Duplicate records, missing or null values are removed, data formats are standardised and data type consistency is ensured so that it is the same across the dataset. The processed data is then stored for optimisation and analytical tasks.

c) **Performance Metrics Module**

The performance metrics module evaluates the optimisation performance of the scraping workflow. This module calculates the processing time, CPU utilisation, memory consumption and throughput performance. These values are used to compare the efficiency of different optimisation techniques and measure the overall system performance during large-scale data scraping and processing activities.

**3. Integration Layer**

The integration layer acts as the bridge between system components to ensure interaction across modules. Within this layer, an API Gateway is implemented to manage incoming and outgoing requests between the application modules and the database layer. The API Gateway

supports high-volume web scraping activities which improves the overall efficiency of the system architecture.

**4. Database Layer**

The database layer is responsible for storing and managing both raw and processed data collected from JobStreet.com. The Main Database stores cleaned and transformed datasets for further analysis and visualisation. Google Colab is utilised as a cloud-based environment to host datasets and support large-scale processing activities. The API Endpoint provides controlled access to stored data for external services or future system expansion. This layer ensures organised data management, efficient retrieval and reliable storage of large datasets generated through the web scraping process.

## 2.2 Tools and Frameworks Used

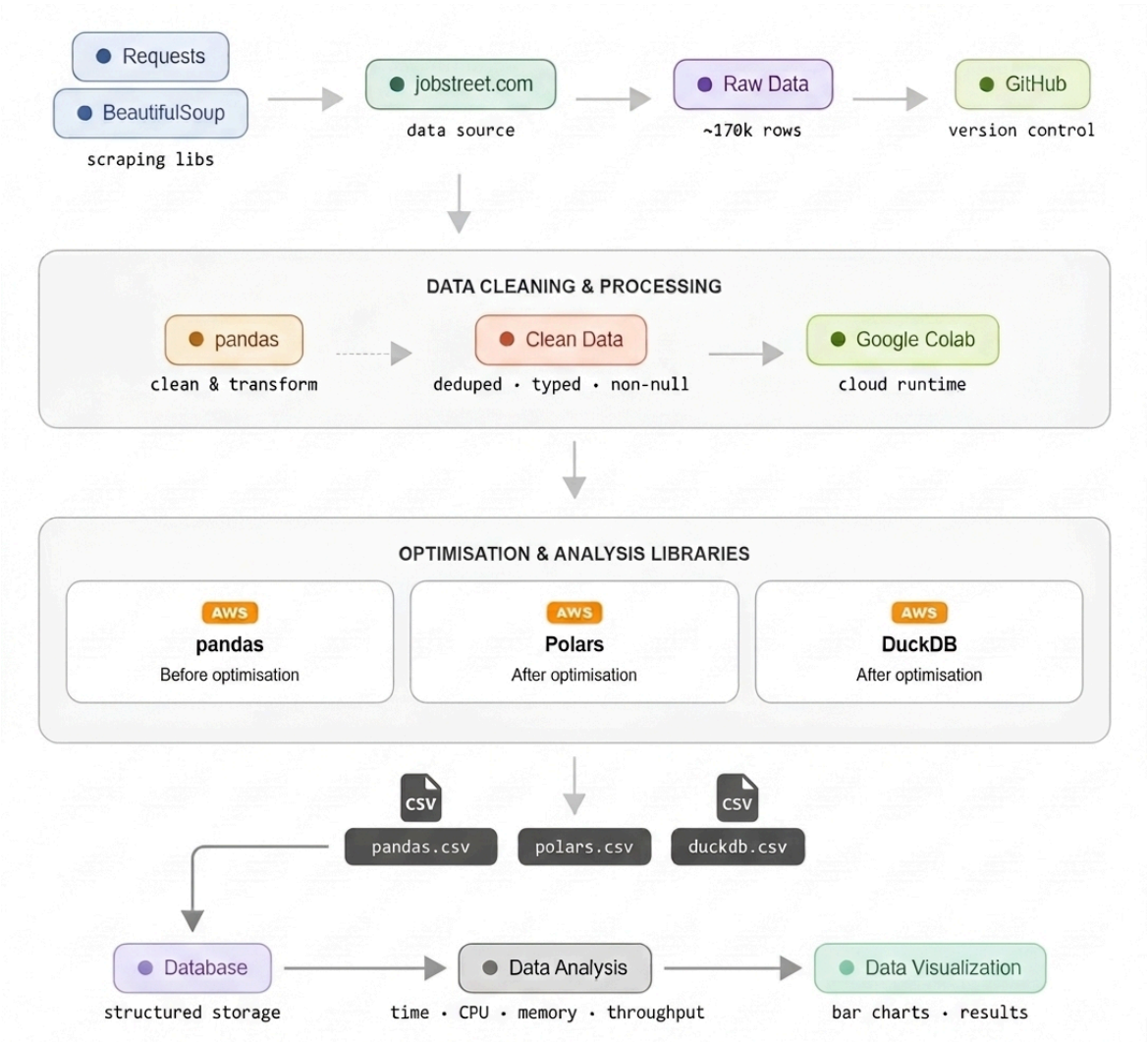



***Figure 2 :*** *Architecture of Tools and Frameworks Used*

Figure 2 shows the overall workflow and system architecture of a web scraping and data analysis pipeline for JobStreet.com. The process begins with the Requests and BeautifulSoup libraries, which are used to scrape data from JobStreet.com. The scraped data is stored as Raw Data containing approximately 130,000 rows and is uploaded in GitHub. Then, the data is cleaned and processed using the pandas library to transform the dataset by removing duplicates, handling missing values, and standardising data types. The cleaned data is then processed in Google Colab, which provides a cloud runtime environment for handling large-scale datasets. Next, the data is optimised and analysed, where different libraries such as pandas, Polars, and DuckDB are tested and compared to evaluate optimisation performance. Finally, the processed data is stored in a database which is then used for data analysis. Data analysis is handled by analysing and comparing processing time, CPU usage, memory usage, and throughput. Data visualisation such as charts and graphical outputs are used to present the analysis.

### 2.3 Roles of Team Members

| Member Name | Task |
|---|---|
| Najma Shakirah Binti Shahrulzaman | Data cleaning, Documentation for Introduction, Data Processing and Conclusion |
| Nurul Asyikin Binti Khairul Anuar | Data Optimization using Pandas, Polars, DuckDB, Documentation for Optimization Techniques, Performance Evaluation and Challenges and Limitations |
| Harini A/P Sangaran | Web crawling, Documentation for System Design and Architecture and Data Collection |

***Table 1 :*** *Tasks Assigned to Each Member*

## 3.0 Data Collection

This chapter explains the methodology used to collect job listing data from JobStreet.com through web scraping techniques. It describes the crawling process, pagination handling, retry

mechanisms, and rate-limiting strategies implemented during data extraction. The chapter also presents the dataset collected and discusses the ethical considerations followed throughout the scraping process.

## 3.1 Crawling method

A synchronous web crawler was developed using Python to scrape job listings from Jobstreet Malaysia and Jobstreet Singapore across various countries which are Malaysia, Singapore, Indonesia, Thailand and Philippines. The crawler performs a keyword-based page-by-page extraction process using Python libraries such as requests, BeautifulSoup, time, random, and pandas. The crawler operates sequentially without asynchronous processing or multiprocessing.

### 3.1.1 Synchronous Architecture

The crawler uses a synchronous crawling architecture where each request and extraction process is completed before moving to the next page. The synchronous workflow consists of several stages. The crawler begins with a predefined keyword and country combination. A request URL is dynamically generated using the keyword slug and page number. Then, a HTTP GET request is issued to retrieve the HTML content of the page. The crawler waits for the response before continuing to the next step. If the request fails, a retry mechanism retries the request up to three times before skipping the page. Once a successful response is received, the HTML content is parsed using BeautifulSoup. Relevant job attributes such as country, job title, company name, salary, location, posted date, employment type, and URL are extracted. Then, the extracted records are appended into a list and periodically saved into a CSV file. Finally, the crawler proceeds to the next page after completing extraction for the current page. This sequential crawling strategy ensures execution and easier debugging while minimizing excessive request rates that may lead to HTTP 403 rate-limiting errors.

### 3.1.2 Pagination

Pagination was implemented through iterative page navigation. The crawler starts from page 1 and continuously increments the page number until multiple consecutive empty pages are encountered.

```
keyword_slug = (
    keyword
    .replace(" ", "-")
    .lower()
)

url = (
    f"{domain}/"
    f"{keyword_slug}-jobs"
    f"?page={page}"
)
```

The keywords used to crawl data from jobstreet.com are python, java, software engineer, data analyst, cloud engineer, accounting, finance, marketing, healthcare and so on. This keyword-based pagination allows the web crawler to bypass pagination limitations commonly found in generic search pages and allows collection of a larger number of job listings.

The crawling process stops automatically when three consecutive pages return zero job listings.

```
if count == 0:

    consecutive_empty += 1

    if (
        consecutive_empty
        >= MAX_EMPTY_PAGES
    ):

        break
```

### 3.1.3 Rate Limiting and Retry Logic

To minimize server overload and reduce the risk of request blocking, the crawler incorporates rate-limiting and retry mechanisms. The crawler uses randomized delays between requests:

```
time.sleep(
    random.uniform(
        SLEEP_MIN,
```

```
    SLEEP_MAX
  )
)
```

The delay range is configured as:

```
SLEEP_MIN = 2
SLEEP_MAX = 5
```

This creates human-like browsing intervals and helps reduce aggressive traffic patterns. Moreover, a retry mechanism is implemented to handle temporary failures such as connection errors, timeouts, or HTTP 403 responses. If a request fails, the crawler retries up to three times before abandoning the page. Failed attempts are logged for monitoring purposes. To further reduce blocking risks, the crawler pauses for 60 seconds after every 100 pages.

```
for attempt in range(1, MAX_RETRIES + 1):

  try:

    response = requests.get(
      url,
      headers=get_headers(),
      timeout=30
    )

    if response.status_code == 200:

      return response.text

MAX_RETRIES = 3

if page % 100 == 0:

  time.sleep(60)
```

**3.1.4 Sequential Data Fetch and Parsing**

After receiving the HTML response, the crawler parses the page using BeautifulSoup. Then, the crawler locates all job cards using soup.find_all().

```
soup = BeautifulSoup(
    html,
    "html.parser"
)

job_cards = soup.find_all("article")
```

For every job card identified, the crawler extracts several structured fields which are Country, Job title, Company name, Location, Salary, Posted date, Employment type and URL.

Example extraction logic :

```
title_tag = card.find(
    "a",
    attrs={
        "data-automation":
        "jobTitle"
    }
)

title = title_tag.get_text(
    strip=True
)

Salary extraction:

salary_tag = card.find(
    attrs={
        "data-automation":
        "job-card-salary"
    }
)

Company extraction:

company_tag = card.find(
    attrs={
        "data-automation":
        "advertiser-name"
```

```
    }
)
```

Each extracted record is appended into a Python list :

```
jobs.append({

    "Country": country,
    "Keyword": keyword,
    "Title": title,
    "Company": company,
    "Location": location,
    "Salary": salary,
    "Posted": posted,
    "Employment Type": employment,
    "URL": link
})
```

The accumulated data is periodically converted into a pandas DataFrame and exported into CSV format. Duplicate job listings are removed using URL-based deduplication.

```
df = df.drop_duplicates(
    subset=["URL"]
)
```

**3.2 Number of Records Collected**

After executing the full crawl across multiple countries and keywords, the crawler successfully collects thousands of job listings from JobStreet platforms across Southeast Asia. The crawler extracted job listings from Malaysia, Singapore, Indonesia, Thailand and Philippines. All extracted listings were saved into CSV format for preprocessing and further analysis.

| Data | Description |
|---|---|
| Country | Country where the job is listed |
| Title | Job title or role name |
| Company | Company or employer name |

| | |
|---|---|
| Location | Job location or city |
| Salary | Salary information if available |
| Posted | Job posting date |
| Employment_type | Type of employment such as full-time or contract |
| URL | Direct link to the job listing |

***Table 2 :*** *Attributes Collected from Web Scraping*

### 3.3 Ethical Considerations

Ethical compliance was considered throughout the scraping process to ensure responsible data collection.

#### 3.3.1 Respecting Server Load

Several measures were implemented to minimize impact on the target servers which are :

- Randomized request delays were used between requests.
- Long cooldown intervals were introduced after every 100 pages.
- The crawler uses synchronous execution instead of aggressive parallel crawling.
- Retry limits prevent excessive repeated requests.

#### 3.3.2 Data Scope and Privacy

Only publicly accessible job listings were collected. The crawler did not require login authentication, did not access hidden APIs, did not scrape personal user information and did not collect sensitive data such as phone numbers, emails, or IP addresses. The extracted information consisted only of publicly displayed job advertisement content.

#### 3.3.3 Legal Review and Transparency

The crawler used a custom browser header to identify itself as a standard web client. The scraping activity was conducted strictly for academic and educational purposes. Data collection was limited to publicly available information and was not redistributed commercially. The resulting dataset was used solely for coursework, analytics, and research-related activities within the project scope.

```
headers = {

   "User-Agent":
   random.choice(USER_AGENTS),

   "Accept-Language":
   "en-US,en;q=0.9",

   "Referer":
   "https://www.google.com/"
}
```

**4.0 Data Processing**

After the data extraction and collection has completed, the raw dataset undergoes the data cleaning and transformation process. This is to ensure that the final dataset is clean, analytically consistent for performance benchmarking in the next phase. The following subsections explains each processing step in the order it was applied.

**4.1 Cleaning Methods**

The raw dataset consists of 122,767 rows and 9 columns which are Country, Title, Company, Location, Salary, Posted, Employment Type, Industry, URL. After loading the raw data, an initial inspection is conducted as shown in Figure 4.1

```
print('=== Data Types ===')
print(df.dtypes)

print('\n=== Missing Values ===')
missing = df.isnull().sum()
pct     = (missing / len(df) * 100).round(2)
print(pd.DataFrame({'Missing': missing, 'Pct (%)': pct}))

print(f'\n=== Duplicate Rows ===\nDuplicates: {df.duplicated().sum():,}')
```

```
=== Data Types ===
Country            object
Title              object
Company            object
Location           object
Salary             object
Posted             object
Employment Type    object
Industry           object
URL                object
dtype: object

=== Missing Values ===
                  Missing  Pct (%)
Country                 0     0.00
Title                   0     0.00
Company                 0     0.00
Location                0     0.00
Salary              69817    56.87
Posted                  0     0.00
Employment Type       379     0.31
Industry            28632    23.32
URL                     0     0.00

=== Duplicate Rows ===
Duplicates: 9,760
```

***Figure 4.1:*** *Initial Inspection of JobStreet Raw Dataset*

The output shows that the data type for all columns are object(string type). Three columns exhibited missing values which are *salary* (69817 missing records), *employment type* (379 missing records) and *industry* (28632 missing records). The output also shows that there are 9,760 duplicate data.

## 4.2 Data Transformation

### 4.2.1 Column Standardization

Column names were standardised by stripping whitecase, converting it to lowercase and replacing spaces with underscores (e.g., Employment Type to employment_type). This is done to prevent errors when running the pipeline.

```
df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(' ', '_', regex=False)
)
print('Columns:', df.columns.tolist())
```

```
Columns: ['country', 'title', 'company', 'location', 'salary', 'posted', 'employment_type', 'industry', 'url']
```

***Figure 4.2 :*** *Column Name Standardisation*

### 4.2.2 Remove Duplicate Records

All duplicate rows were removed using the drop_duplicates() function. After this step, the dataset was reduced to 113,007 unique rows, and the index was reset.

```
before = len(df)
df = df.drop_duplicates()
df = df.reset_index(drop=True)
print(f'Removed {before - len(df):,} duplicate rows → {len(df):,} remaining')
```

```
Removed 9,760 duplicate rows → 113,007 remaining
```

***Figure 4.3 :*** *Drop Duplicates Records*

### 4.2.3 Text Field Normalization

All string columns were normalized to remove leading and trailing whitespace using str.strip() function. The *title* column was cleaned to remove multiple whitespace into a single space and remove non-printable characters (e.g., \t) as shown in Figure 4.4. The code snippet in Figure 4.5 replaces empty strings with NaN in *company* column and then fill the NaN values with 'Unknown'. It then prints the number of unique companies and the top 10 frequent companies names in the job listing. The same steps are applied for other string fields which is *location*, *employment_type* and *industry* field. Although the *company* and *location* column has no missing records, the same processing steps were applied as precaution and ability to handle missing data if new data is appended.

```python
df['title'] = (
    df['title']
    .astype(str)
    .str.strip()
    # Collapse multiple spaces
    .str.replace(r'\s+', ' ', regex=True)
    # Remove non-printable chars (keep unicode letters, digits, punctuation)
    .str.replace(r'[\x00-\x1f\x7f]', '', regex=True)
)
df.loc[df['title'].isin(['', 'nan']), 'title'] = np.nan
print(f"title nulls: {df['title'].isna().sum():,}")
print(df['title'].head(5))
```

```
title nulls: 0
0       Assistant Jewellery Sales Manager (Malaysia) 珠寶銷售副經理 (馬來西亞)
1                                    Cashier (Malaysia) 出納員 (馬來西亞)
2    Assistant Jewellery Sales Supervisor (Malaysia) 珠寶銷售副主任 (馬來西亞)
3                                          Big Data Hadoop Developer
4               Jewellery Sales Consultant (Malaysia) 珠寶銷售顧問 (馬來西亞)
Name: title, dtype: object
```

***Figure 4.4 :*** *title Column Normalization*

```python
df['company'] = (
    df['company']
    .astype(str)
    .str.strip()
    .str.replace(r'\s+', ' ', regex=True)
)
df.loc[df['company'].isin(['', 'nan']), 'company'] = np.nan
df['company'] = df['company'].fillna('Unknown')
print(f"Distinct companies: {df['company'].nunique():,}")
print(df['company'].value_counts().head(10))
```

```
Distinct companies: 38,010
company
Private Advertiser               4166
PERSOL                           1038
SmartHire by SEEK                 559
Emapta                            363
Ideals Recruitment Pte Ltd        316
PT. BFI FINANCE INDONESIA, Tbk    290
AlwaysHired Pte. Ltd.             287
RecruitFirst Pte. Ltd             283
Foundever®                        277
PERSOL Thailand                   263
Name: count, dtype: int64
```

***Figure 4.5 :*** *company Column Normalization*

```python
df['location'] = (
    df['location']
    .astype(str)
    .str.strip()
    .str.title()
    .str.replace(r'\s+', ' ', regex=True)
)
df.loc[df['location'].isin(['', 'Nan']), 'location'] = np.nan
df['location'] = df['location'].fillna('Unknown')
print(f"Distinct locations: {df['location'].nunique():,}")
print(df['location'].value_counts().head(10))
```

```
Distinct locations: 2,801
location
Singapore         5106
Bangkok           4476
Kuala Lumpur      4038
Central Region    3350
Makati City       2816
Jakarta           2752
South Jakarta     1795
Metro Manila      1765
Quezon City       1613
Petaling Jaya     1403
Name: count, dtype: int64
```

***Figure 4.6 :*** *location Column Normalization*

```python
df['employment_type'] = (
    df['employment_type']
    .astype(str)
    .str.strip()
    .str.title()
)
df.loc[df['employment_type'].isin(['', 'Nan']), 'employment_type'] = np.nan
df['employment_type'] = df['employment_type'].fillna('Unknown')

print(df['employment_type'].value_counts())
```

```
employment_type
Full Time     101609
Contract        9444
Part Time       1555
Unknown          351
Internship        44
Temporary          4
Name: count, dtype: int64
```

***Figure 4.7 :*** *employment_type Column Normalization*

```
df['industry'] = (
    df['industry']
    .astype(str)
    .str.strip()
    .str.title()
    .str.replace(r'\s+', ' ', regex=True)
)
df.loc[df['industry'].isin(['', 'Nan']), 'industry'] = np.nan
df['industry'] = df['industry'].fillna('Unknown')

print(f"Distinct industries: {df['industry'].nunique():,}")
print(df['industry'].value_counts().head(15))
```

```
Distinct industries: 17
industry
Unknown             26194
Sales               14595
Manufacturing       13596
Engineering         12716
Accounting           6886
Marketing            6620
Retail               6263
Human Resources      4486
Healthcare           4109
Banking              4095
Finance              4025
Construction         3186
Education            2775
Design               2098
Legal                 921
Name: count, dtype: int64
```

***Figure 4.8 :*** *industry Column Normalization*

### 4.2.4 Salary Parsing and Currency Extraction

The salary column was the most technically complex data cleaning challenge as it contained six different currencies from JobStreet's multi-country coverage and had inconsistent formatting. 56.87% of salary column was not available, In Figure 4.9, the salary column is normalized for 'N/A' strings with np.nan. It then creates a salary_disclosed flag which is a boolean column to identify which rows have disclosed salary.

```
# ── Step 1: Normalise all 'n/a'-like strings → NaN ──────────────────────
NA_STRINGS = {'n/a', 'na', 'none', 'null', '-', '--', 'nan', '', 'not specified', 'undisclosed'}

df['salary'] = df['salary'].astype(str).str.strip()
df.loc[df['salary'].str.lower().isin(NA_STRINGS), 'salary'] = np.nan

# ── Step 2: Add a disclosure flag ──────────────────────────────────────
df['salary_disclosed'] = df['salary'].notna()

# ── Step 3: Summary ────────────────────────────────────────────────────
total        = len(df)
has_salary   = df['salary_disclosed'].sum()
no_salary    = total - has_salary

print(f'Total rows        : {total:,}')
print(f'Salary disclosed  : {has_salary:,}  ({has_salary/total*100:.1f}%)')
print(f'Salary NOT shown  : {no_salary:,}  ({no_salary/total*100:.1f}%)  ← normal for job boards')
print()
print('Note: salary_min / salary_max will be NaN for undisclosed rows.')
print('      Use salary_disclosed=True to filter to rows with known salary.')
```

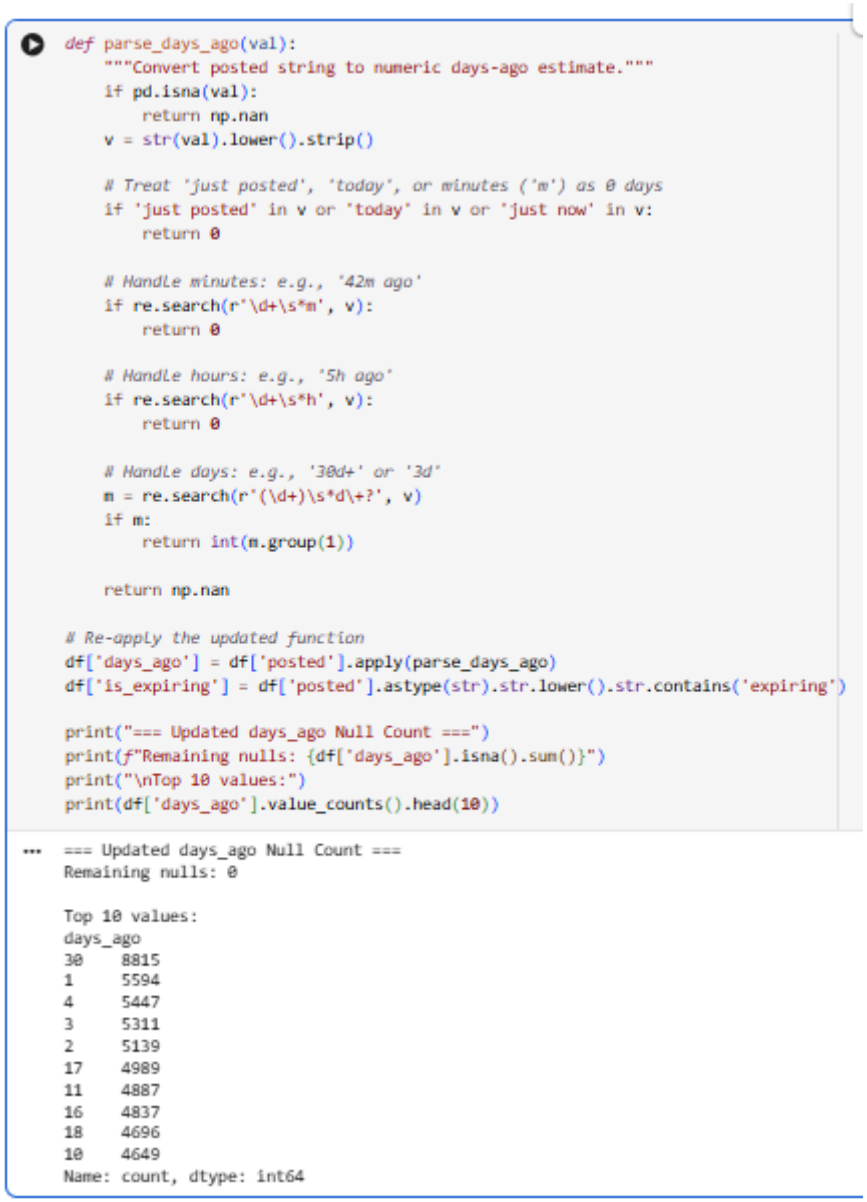
```
Total rows        : 113,007
Salary disclosed  : 48,869  (43.2%)
Salary NOT shown  : 64,138  (56.8%)  ← normal for job boards

Note: salary_min / salary_max will be NaN for undisclosed rows.
      Use salary_disclosed=True to filter to rows with known salary.
```

***Figure 4.9 :*** *salary Column Normalization*

As shown in Appendix A, it parse the salary column from strings like ‘RM 3,000 - RM5,000 per month” to derived columns salary_min, salary_max, salary_currency and salary_period. The code first strip the whitespace and change the contents in the column to lowercase. It also changes any string that does not mention a specific numerical salary value (e.g.,tbd, attractive salary, basic salary) to np.nan. Then the currency for each salary record is identified. The code also detects the different time periods in the code which are monthly, yearly and hourly.

### 4.2.5 Posted Date Parsing

The posted column contain various expressions for date of when is the job listed such as '3d ago', '30d+ ago', '30d+ ago•Expiring', and 'Just posted'. parse_days_ago() function converts the string to number of days since the job was posted.A new boolean column is_expiring was created as a flag to show which listings are expiring. This is shown in Figure 4.10.



```python
def parse_days_ago(val):
    """Convert posted string to numeric days-ago estimate."""
    if pd.isna(val):
        return np.nan
    v = str(val).lower().strip()

    # Treat 'just posted', 'today', or minutes ('m') as 0 days
    if 'just posted' in v or 'today' in v or 'just now' in v:
        return 0

    # Handle minutes: e.g., '42m ago'
    if re.search(r'\d+\s*m', v):
        return 0

    # Handle hours: e.g., '5h ago'
    if re.search(r'\d+\s*h', v):
        return 0

    # Handle days: e.g., '30d+' or '3d'
    m = re.search(r'(\d+)\s*d\+?', v)
    if m:
        return int(m.group(1))

    return np.nan

# Re-apply the updated function
df['days_ago'] = df['posted'].apply(parse_days_ago)
df['is_expiring'] = df['posted'].astype(str).str.lower().str.contains('expiring')

print("=== Updated days_ago Null Count ===")
print(f"Remaining nulls: {df['days_ago'].isna().sum()}")
print("\nTop 10 values:")
print(df['days_ago'].value_counts().head(10))
```

```
=== Updated days_ago Null Count ===
Remaining nulls: 0

Top 10 values:
days_ago
30    8815
1     5594
4     5447
3     5311
2     5139
17    4989
11    4887
16    4837
18    4696
10    4649
Name: count, dtype: int64
```

***Figure 4.10**: Posted Date Parsing*

### 4.2.6 URL Validation

The code snippet in Figure 4.11 shows the validation process of the url column. The code will only keep rows with proper URL (string that begins with ‘http’). The output shows that there are no invalid URLs, so the final number of rows remains the same.

```python
before = len(df)
df = df[df['url'].astype(str).str.startswith('http')]
print(f'Dropped {before - len(df):,} rows with invalid URLs → {len(df):,} remaining')
```

```
Dropped 0 rows with invalid URLs → 113,007 remaining
```

***Figure 4.11:** Validate URLs From url Column*

### 4.3 Data Structure

The final cleaned dataset consists of 113,007 and 16 columns (9 original columns and 7 derived columns). The data structure of the final dataset is shown in Table 4.1.

| Column Name | Origin | Data Type | Description |
|---|---|---|---|
| country | Original | string | Country of the job listing |
| title | Original | string | Job title |
| company | Original | string | Employer name |
| location | Original | string | Job location |
| salary | Original | string | Salary retained for parsed data reference |
| posted | Original | string | Posted retained for parsed data reference |
| employment_type | Original | string | Employment category |
| industry | Original | string | Industry sector |
| url | Original | string | Direct URL link of job listing |
| salary_disclosed | Derived | boolean | True if employer published a salary; False if withheld |
| salary_min | Derived | float64 | Lower bound of salary range parsed from raw string |
| salary_max | Derived | float64 | Upper bound of salary range; equals salary_min for single-value listings |
| salary_currency | Derived | string | Currency code detected from salary string symbol or keyword |
| salary_period | Derived | string | Pay period parsed from keywords |
| days_ago | Derived | float64 | Numeric days since posting parsed from relative date string |
| is_expiring | Derived | boolean | True if the posted field contains '•Expiring' |

***Table 4.1:*** *Data Structure of Cleaned Data*

## 5.0 Optimization Techniques

This section describes the optimization techniques that used in this project. To ensure a fair and consistent comparison, the same cleaned dataset and processing procedure were used for testing all three libraries. The optimization procedure concentrated on enhancing throughput, decreasing memory utilization and increasing processing speed when managing massive data.

During execution, the total processing time, CPU usage, memory usage and throughput which is the number of records processed per second is collected.Then, the metrics for each optimisation stage were stored as separate JSON files for further analysis and visualization.

### 5.1 Pandas Optimization

Pandas was used as the project's baseline optimisation techniques. It is one of the most used Python libraries for data analysis and data processing due to its easy syntax and flexibility. Some of the operations performed are count the total jobs, including the full-time and contracts jobs, identifying employment positions with disclosed salaries and measuring performance metrics throughout execution as shown in Figure 5.1.

Although Pandas is simple to use, it mostly uses single-threaded execution, making it slower when processing huge datasets compared to more optimised libraries.

```python
import pandas as pd
import time
import psutil
import os
import json

def run_pandas_optimization():

    # Start performace tracking
    start_time = time.time()
    process = psutil.Process(os.getpid())
    cpu_start = psutil.cpu_percent(interval=None)
    memory_start = process.memory_info().rss / (1024 * 1024)

    # Read CSV file
    df = pd.read_csv("jobstreet_cleaned.csv")
    df['days_ago'] = pd.to_numeric(df['days_ago'], errors='coerce')

    # Operations
    total_jobs = len(df)
    full_time_jobs = len(df[df['employment_type'] == 'Full Time'])
    contract_jobs = len(df[df['employment_type'] == 'Contract'])
    salary_disclosed_jobs = len(df[df['salary_disclosed'] == True])
    expiring_jobs = len(df[df['is_expiring'] == True])
    avg_days_ago = df['days_ago'].mean()
    most_common_country = df['country'].mode()[0]
    most_common_industry = df['industry'].mode()[0]

    # End of performance tracking
    end_time = time.time()
    execution_time = end_time - start_time
    cpu_end = psutil.cpu_percent(interval=None)
    memory_end = process.memory_info().rss / (1024 * 1024)
    throughput = total_jobs / execution_time

    # Metrics
    metrics = {
        "Optimization Stage": "Pandas Optimization",
        "Total Jobs": total_jobs,
        "Total Full Time Jobs": full_time_jobs,
        "Total Contract Jobs": contract_jobs,
        "Salary Disclosed Jobs": salary_disclosed_jobs,
        "Expiring Jobs": expiring_jobs,
        "Average Days Ago": avg_days_ago,
        "Most Common Country": most_common_country,
        "Most Common Industry": most_common_industry,
        "Total Processing Time (seconds)": execution_time,
        "CPU Usage (%)": cpu_end - cpu_start,
        "Memory Usage (MB)": abs(memory_end - memory_start),
        "Throughput (records/second)": throughput
    }

    # Print the results
    print("\n🐼 Pandas Optimization Complete")
    print(f"📄 Total Jobs: {total_jobs}")
    print(f"💼 Full Time Jobs: {full_time_jobs}")
    print(f"📑 Contract Jobs: {contract_jobs}")
    print(f"💰 Salary Disclosed Jobs: {salary_disclosed_jobs}")
    print(f"⏳ Expiring Jobs: {expiring_jobs}")
    print(f"📅 Average Days Ago: {avg_days_ago:.2f}")
    print(f"🌍 Most Common Country: {most_common_country}")
    print(f"🏢 Most Common Industry: {most_common_industry}")
    print(f"🕒 Processing Time: {execution_time:.4f} seconds")
    print(f"🧠 CPU Usage: {cpu_end - cpu_start:.2f}%")
    print(f"💾 Memory Usage: {abs(memory_end - memory_start):.2f} MB")
    print(f"⚡ Throughput: {throughput:.2f} records/second")

    # Save JSON file
    with open("pandas_metrics.json", "w") as f:
        json.dump(metrics, f)

    return metrics

run_pandas_optimization()
```

***Figure 5.1:*** *The Code Segment for Pandas Optimization*

Figure 5.2 shows the Pandas optimization was implemented. The dataset is processed with standard DtaFrame and the performance metrics such as processing time, CPU usage and memory usage are recorded during the execution.

```
🐼 Pandas Optimization Complete
📄 Total Jobs: 113007
💼 Full Time Jobs: 101609
📑 Contract Jobs: 9444
💰 Salary Disclosed Jobs: 48869
⏳ Expiring Jobs: 14831
📅 Average Days Ago: 14.48
🌍 Most Common Country: Malaysia
🏢 Most Common Industry: Unknown
🕒 Processing Time: 1.2018 seconds
🧠 CPU Usage: 65.50%
💾 Memory Usage: 84.57 MB
⚡ Throughput: 94030.83 records/second
{'Optimization Stage': 'Pandas Optimization',
 'Total Jobs': 113007,
 'Total Full Time Jobs': 101609,
 'Total Contract Jobs': 9444,
 'Salary Disclosed Jobs': 48869,
 'Expiring Jobs': 14831,
 'Average Days Ago': np.float64(14.479372725421268),
 'Most Common Country': 'Malaysia',
 'Most Common Industry': 'Unknown',
 'Total Processing Time (seconds)': 1.201807975769043,
 'CPU Usage (%)': 65.5,
 'Memory Usage (MB)': 84.57421875,
 'Throughput (records/second)': 94030.82878334723}
```

***Figure 5.2:*** *Output of Pandas Optimization*

Figure 5.2 shows the output generated by Pandas. The total processing time recorded is longer than the optimized approaches. The output demonstrates that Pandas is functional yet inefficient for large-scale processing.

### 5.2 Polars Optimization

Polars was used to improve data processing performance over Pandas since it is faster when handling with large datasets. The same processing methods were used to ensure fair comparison, such as filtering, grouping and calculating performance metrics.

```python
import polars as pl
import time
import psutil
import os
import json

def run_polars_optimization():

    # Start performace tracking
    start_time = time.time()
    process = psutil.Process(os.getpid())
    cpu_start = psutil.cpu_percent(interval=None)
    memory_start = process.memory_info().rss / (1024 * 1024)

    # Read CSV file
    df = pl.read_csv(
        "jobstreet_cleaned.csv",
        ignore_errors=True,
        null_values=["", "NA", "N/A"]
    )
    df = df.with_columns([
        pl.col("days_ago").cast(pl.Float64, strict=False)
    ])

    # Operations
    total_jobs = df.height
    full_time_jobs = df.filter(
        pl.col("employment_type") == "Full Time"
    ).height
    contract_jobs = df.filter(
        pl.col("employment_type") == "Contract"
    ).height
    salary_disclosed_jobs = df.filter(
        pl.col("salary_disclosed") == True
    ).height
    expiring_jobs = df.filter(
        pl.col("is_expiring") == True
    ).height
    avg_days_ago = df["days_ago"].mean()
    most_common_country = (
```

```python
    # Operations
    total_jobs = df.height
    full_time_jobs = df.filter(
        pl.col("employment_type") == "Full Time"
    ).height
    contract_jobs = df.filter(
        pl.col("employment_type") == "Contract"
    ).height
    salary_disclosed_jobs = df.filter(
        pl.col("salary_disclosed") == True
    ).height
    expiring_jobs = df.filter(
        pl.col("is_expiring") == True
    ).height
    avg_days_ago = df["days_ago"].mean()
    most_common_country = (
        df.group_by("country")
        .len()
        .sort("len", descending=True)
        .row(0)[0]
    )
    most_common_industry = (
        df.group_by("industry")
        .len()
        .sort("len", descending=True)
        .row(0)[0]
    )

    # End of performance tracking
    end_time = time.time()
    execution_time = end_time - start_time
    cpu_end = psutil.cpu_percent(interval=None)
    memory_end = process.memory_info().rss / (1024 * 1024)
    throughput = total_jobs / execution_time
```

```python
    # Metrics
    metrics = {
        "Optimization Stage": "Polars Optimization",
        "Total Jobs": total_jobs,
        "Total Full Time Jobs": full_time_jobs,
        "Total Contract Jobs": contract_jobs,
        "Salary Disclosed Jobs": salary_disclosed_jobs,
        "Expiring Jobs": expiring_jobs,
        "Average Days Ago": avg_days_ago,
        "Most Common Country": most_common_country,
        "Most Common Industry": most_common_industry,
        "Total Processing Time (seconds)": execution_time,
        "CPU Usage (%)": cpu_end - cpu_start,
        "Memory Usage (MB)": abs(memory_end - memory_start),
        "Throughput (records/second)": throughput
    }

    # Print the results
    print("\n Polars Optimization Complete")
    print(f" Total Jobs: {total_jobs}")
    print(f" Full Time Jobs: {full_time_jobs}")
    print(f" Contract Jobs: {contract_jobs}")
    print(f" Salary Disclosed Jobs: {salary_disclosed_jobs}")
    print(f" Expiring Jobs: {expiring_jobs}")
    print(f" Average Days Ago: {avg_days_ago:.2f}")
    print(f" Most Common Country: {most_common_country}")
    print(f" Most Common Industry: {most_common_industry}")
    print(f" Processing Time: {execution_time:.4f} seconds")
    print(f" CPU Usage: {cpu_end - cpu_start:.2f}%")
    print(f" Memory Usage: {abs(memory_end - memory_start):.2f} MB")
    print(f" Throughput: {throughput:.2f} records/second")

    # Save JSON file
    with open("polars_metrics.json", "w") as f:
        json.dump(metrics, f)

    return metrics

run_polars_optimization()
```

***Figure 5.3**: The Code Segment for Polars Optimization*

Figure 5.3 shows the Polars code used to process the dataset and calculate performance metrics.

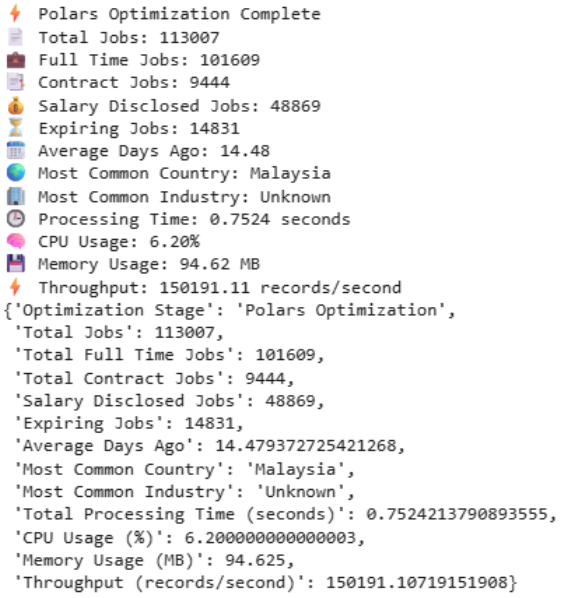



***Figure 5.4**: Output of Polars Optimization*

Figure 5.4 displays the results after execution of Polars. It is obvious that Polars performs better when handling big datasets than Pandas, showing better performance in handling large datasets.

### 5.3 DuckDB Optimization

Another optimization technique employed in this project was DuckDB. It is a SQL-based engine that effectively handles big datasets and is built for quick analytical processing. To provide a fair comparison with Pandas and Polars, the same dataset and processing logic were employed that was shown in Figure 5.5.

```python
import duckdb
import time
import psutil
import os
import json

def run_duckdb_optimization():

    # Start performance tracking
    start_time = time.time()
    process = psutil.Process(os.getpid())
    cpu_start = psutil.cpu_percent(interval=None)
    memory_start = process.memory_info().rss / (1024 * 1024)

    # Connect to DuckDB
    conn = duckdb.connect(database=':memory:')

    # Query
    result = conn.execute("""
        SELECT
            COUNT(*) AS total_jobs,

            SUM(
                CASE
                    WHEN employment_type = 'Full Time'
                    THEN 1
                    ELSE 0
                END
            ) AS full_time_jobs,

            SUM(
                CASE
                    WHEN employment_type = 'Contract'
                    THEN 1
                    ELSE 0
                END
            ) AS contract_jobs,

            SUM(
                CASE
                    WHEN salary_disclosed = TRUE
                    THEN 1
                    ELSE 0
                END
            ) AS salary_disclosed_jobs,

            SUM(
                CASE
                    WHEN is_expiring = TRUE
                    THEN 1
                    ELSE 0
                END
            ) AS expiring_jobs,

            AVG(TRY_CAST(days_ago AS DOUBLE)) AS avg_days_ago,
            MODE(country) AS most_common_country,
            MODE(industry) AS most common industry
             MODE(country) AS most_common_country,
             MODE(industry) AS most_common_industry
        FROM read_csv_auto('jobstreet_cleaned.csv')
    """).fetchone()

    total_jobs = result[0]
    full_time_jobs = result[1]
    contract_jobs = result[2]
    salary_disclosed_jobs = result[3]
    expiring_jobs = result[4]
    avg_days_ago = result[5]
    most_common_country = result[6]
    most_common_industry = result[7]

    # End of performance tracking
    end_time = time.time()
    execution_time = end_time - start_time
    cpu_end = psutil.cpu_percent(interval=None)
    memory_end = process.memory_info().rss / (1024 * 1024)
    throughput = total_jobs / execution_time

    metrics = {
        "Optimization Stage": "DuckDB Optimization",
        "Total Jobs": total_jobs,
        "Total Full Time Jobs": full_time_jobs,
        "Total Contract Jobs": contract_jobs,
        "Salary Disclosed Jobs": salary_disclosed_jobs,
        "Expiring Jobs": expiring_jobs,
        "Average Days Ago": avg_days_ago,
        "Most Common Country": most_common_country,
        "Most Common Industry": most_common_industry,
        "Total Processing Time (seconds)": execution_time,
        "CPU Usage (%)": cpu_end - cpu_start,
        "Memory Usage (MB)": abs(memory_end - memory_start),
        "Throughput (records/second)": throughput
    }

    # Print the results
    print("\n DuckDB Optimization Complete")
    print(f" Total Jobs: {total_jobs}")
    print(f" Full Time Jobs: {full_time_jobs}")
    print(f" Contract Jobs: {contract_jobs}")
    print(f" Salary Disclosed Jobs: {salary_disclosed_jobs}")
    print(f" Expiring Jobs: {expiring_jobs}")
    print(f" Average Days Ago: {avg_days_ago:.2f}")
    print(f" Most Common Country: {most_common_country}")
    print(f" Most Common Industry: {most_common_industry}")
    print(f" Processing Time: {execution_time:.4f} seconds")
    print(f" CPU Usage: {cpu_end - cpu_start:.2f}%")
    print(f" Memory Usage: {abs(memory_end - memory_start):.2f} MB")
    print(f" Throughput: {throughput:.2f} records/second")

    # Save the JSON file
    with open("duckdb_metrics.json", "w") as f:
        json.dump(metrics, f)

    conn.close()

    return metrics

run_duckdb_optimization()
```

***Figure 5.5**: The Code Segment for DuckDB Optimization*

Figure 5.5 shows the SQL-based DuckDB code used to process the dataset and compute performance metrics.


***Figure 5.6:*** *Output of DuckDB Optimization*

Figure 5.6 displays the results of performing DuckDB. It can be seen that the DuckDB outperforms Pandas in terms of processing times and resource usage, as well as performance.

### 5.4 Summary of Optimization Techniques

In this project, three optimization approaches were used to improve the performance of the data processing process which is Pandas, Polars and DuckDB. These tools were chosen to demonstrate several ways to manage large datasets from using a standard Python library to more complex analytical engines.

Each technique was used the same cleaned JobStreet dataset to maintain consistency in the processing processes. Pandas was chosen as the baseline since it is simple and widely used for data analysis jobs. Then, in order to make faster processing and better resource management, Polars and DuckDB were added to handle larger amounts of data more effectively. In general, these three methods provide good structure for comparing various data processing optimization techniques.

## 6.0 Performance Evaluation

This section evaluates the performance of the optimization techniques used in this project which is Pandas, Polars and DuckDB. The evaluation aimed to compare the processing performance of each techniques when dealing with the cleaned JobStreet dataset. Several test were performed and the results are presented as tables and bar charts for easier comparison and analysis.

### 6.1 Before and After Optimization

Pandas was chosen as the main library for processing the JobStreet dataset at the beginning of the project because it is easier to understand and more often used in Python for data analysis. However, after texting the processing performance on a dataset with over 100000 records, the execution time and memory usage were still quite high.

To improve overall performance, two additional optimization libraries which is the DuckDB and Polars were tested. DuckDB was chosen because it can efficiently process analytical queries written in SQL syntax while Polars where chosen because it can analyzed huge datasets faster and manage the memory usage more effectively than Pandas.

### 6.2 Testing Environment and Metrics Collection

To make it as fair comparison, all optimization techniques were tested on the same cleaned JobStreet dataset. Moreover, the same processing logic and analysis steps were used for Pandas, Polars and also the DuckDB.

To decrease the inconsistency of the runs, each optimization process were repeated three times and the average result was calculated later. Several Python libraries were used during the evaluation process such as:

- time: to monitor the processing time
- psutil: to measure CPU and memory usage
- json: to store the performance metrics

The collected performance metrics were later used to compare the efficiency and processing performance of Pandas, Polars and also the DuckDB

### 6.3 Comparative Analysis of Performance Metrics

To determine the effectiveness of each optimization techniques, the Pandas, Polars and DuckDB were tested three times with the same dataset and processing steps.Four performance metrics were recorded for each run which is the total processing time, CPU usage, memory usage and also the throughput which is the number of records processed per second.

Next, the average values from the three runs were calculated to make the comparison more reliable and consistent. The results are provided in **tables and figures** below to show the performance differences between the three optimization techniques.

### 6.3.1 Total Processing Time

Table 1 shows the processing time recorded for each optimization technique across three different runs.

| Optimization Stage | Run 1 | Run 2 | Run 3 | Average |
|---|---|---|---|---|
| Pandas Optimization | 0.7013 | 0.6072 | 0.6084 | 0.64 |
| Polars Optimization | 0.1452 | 0.1441 | 0.1421 | 0.14 |
| DuckDB Optimization | 0.2067 | 0.2016 | 0.2042 | 0.20 |

***Table 6.1:*** *Total Processing Time (seconds)*

Based on Table 6.1, Polars had the lowest processing time among other optimization technique, with an average of 0.14 seconds. Pandas had the longest processing time, whereas DuckDB performed in the middle of Pandas and Polars.

Figure 6.1 shows the average processing time for each optimization strategy as a bar chart. According to Figure 6.1, Polars performed the fastest, whereas Pandas took the most processing time.

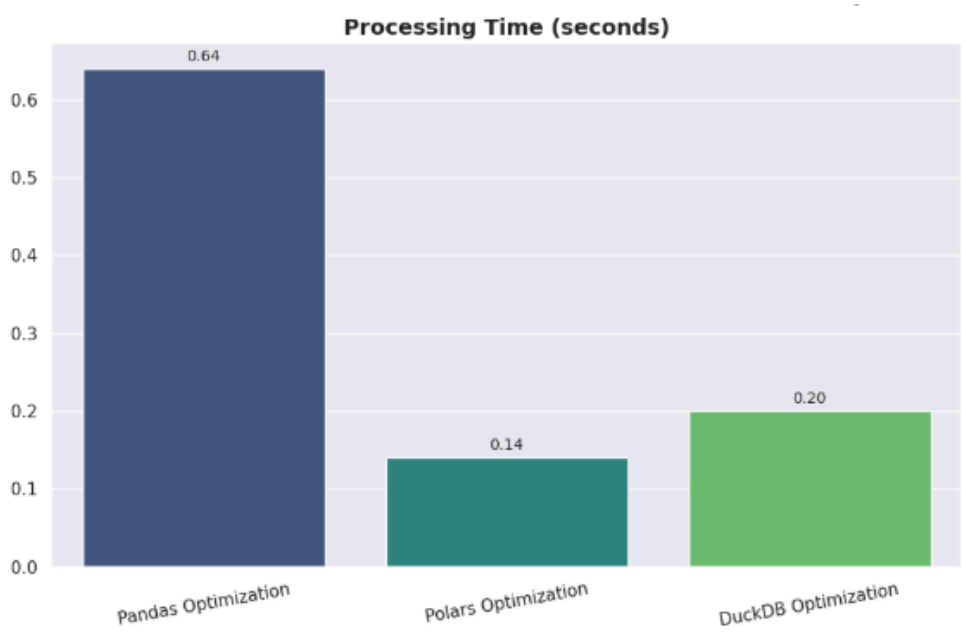



***Figure 6.1:*** *Bar Chart for Processing Time (s)*

### 6.3.2 CPU Usage

Table 6.2 presents the CPU usage percentage for each optimization techniques during execution.

| Optimization Stage | Run 1 | Run 2 | Run 3 | Average |
|---|---|---|---|---|
| Pandas Optimization | 49.91 | 49.41 | 50.95 | 50.09 |
| Polars Optimization | 92.97 | 97.18 | 95.01 | 95.06 |
| DuckDB Optimization | 70.16 | 69.44 | 71.01 | 70.20 |

***Table 6.2:*** *CPU Usage (%)*

Based on the Table 6.2, Polars had the highest CPU utilization with an average of 95.06%, followed by DuckDB and Pandas. Higher CPU utilization demonstrates that Polars optimized system resources to achieve faster processing performance.

Figure 6.2 shows the average CPU consumption for each optimization strategy in the form of a bar chart. The graph clearly shows that Polars used more CPU usage than DuckDB and Pandas.

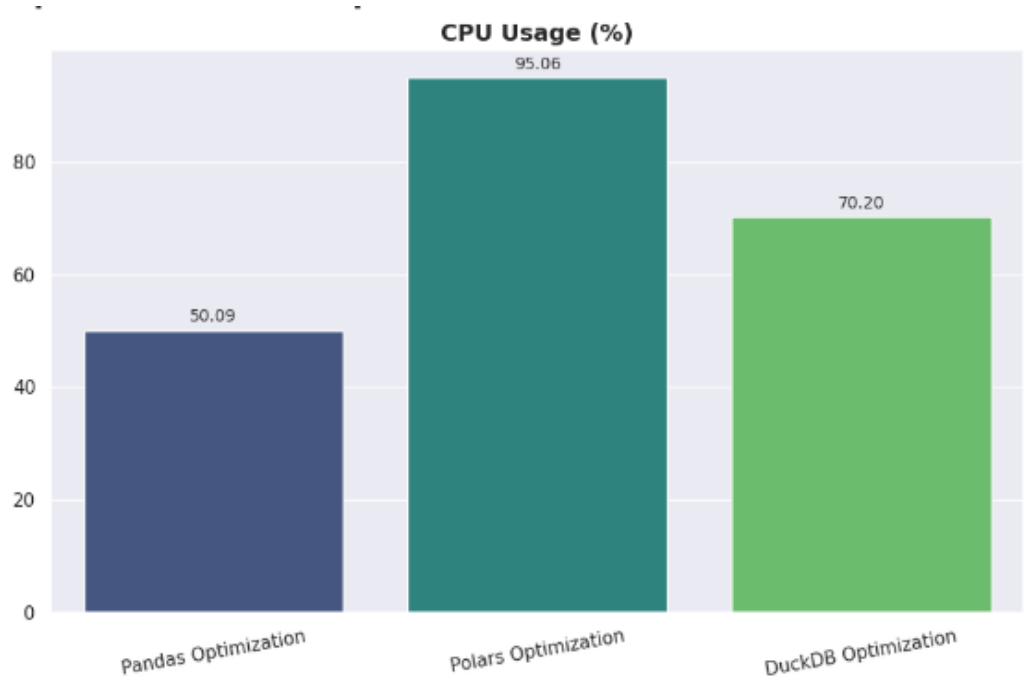



***Figure 6.2:*** *Bar Chart for CPU Usage (%)*

### 6.3.3 Memory Usage

Table 3 shows the memory consumption for each optimization techniques.

| Optimization Stage | Run 1 | Run 2 | Run 3 | Average |
|---|---|---|---|---|
| Pandas Optimization | 812.00 | 818.52 | 818.53 | 816.35 |
| Polars Optimization | 817.89 | 817.90 | 817.90 | 817.90 |
| DuckDB Optimization | 787.23 | 787.05 | 787.11 | 787.13 |

***Table 6.3:*** *Memory Usage (MB)*

According to Table 6.3, DuckDB had the lowest average memory use which is 787.13MB. Meanwhile, Pandas and Polars used slightly more memory during execution. Figure 6.3 shows the average memory use for each optimization strategy as a bar chart. Based on Figure 6.3, DuckDB is the most memory-efficient of the three optimization techniques.

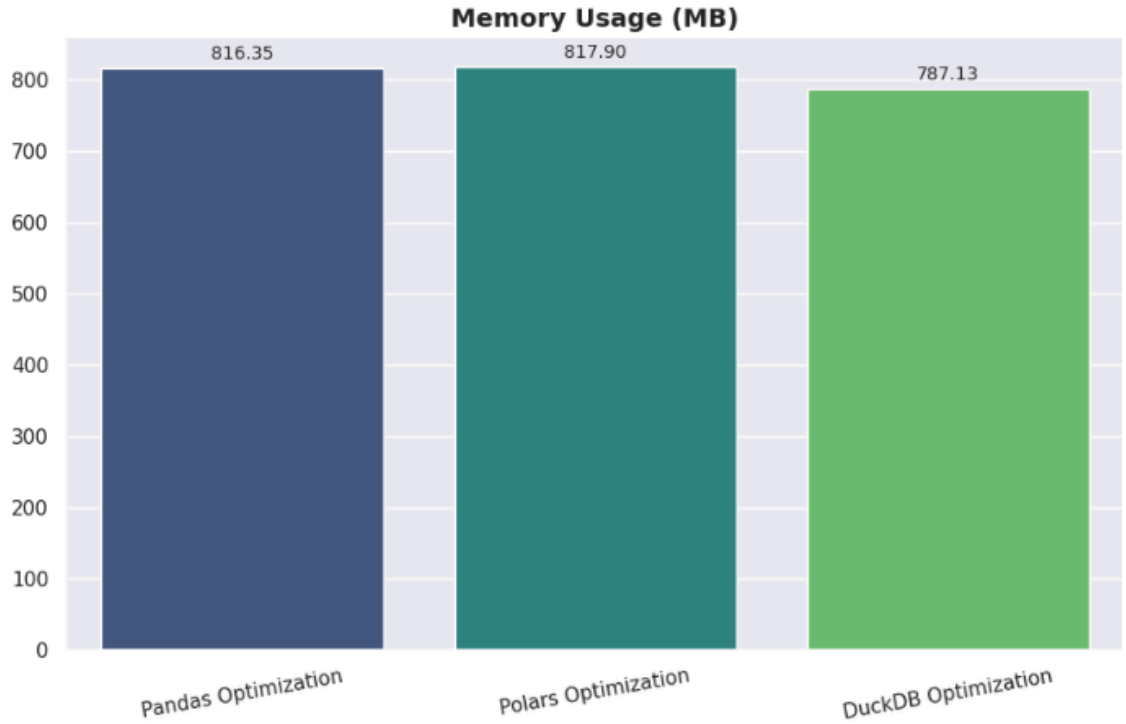



***Figure 6.3:*** *Bar Chart for Memory Usage (MB)*

### 6.3.4 Throughput

Table 4 presents the throughput performance measured in records processed per second.

| Optimization Stage | Run 1 | Run 2 | Run 3 | Average |
|---|---|---|---|---|
| Pandas Optimization | 170728.51 | 185623.50 | 184172.75 | 180174.92 |

| | | | | |
|---|---|---|---|---|
| Polars Optimization | 766843.09 | 805061.70 | 820782.91 | 797562.56 |
| DuckDB Optimization | 523918.85 | 587682.25 | 576896.56 | 562832.55 |

***Table 6.4:*** *Throughput (records/seconds)*

Based on Table 6.4, Polars had the highest throughput, with an average of 797562.56 records handled per second. DuckDB also outperformed Pandas in terms of throughput. Next, Figure 6.4 shows the average throughput for each optimization strategy in a bar chart. According to Figure 6.4, Polars processed the most records per second, showing better overall processing efficiency.

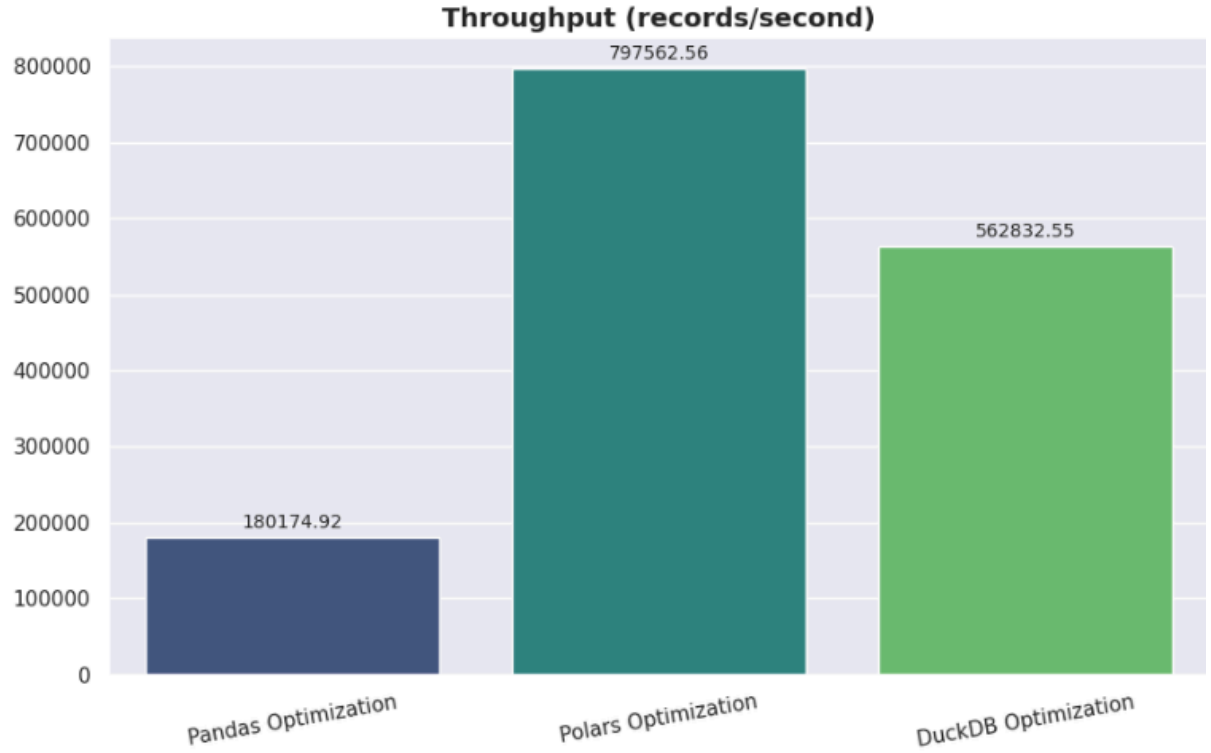



***Figure 6.4:*** *Bar Chart for Throughput (records/second)*

**6.4 Summary of Performance Evaluation**

In general, the Polars and DuckDB outperformed the Pandas in processing the JobStreet dataset. Polars obtained the fastest processing time and the best throughput based on the average results from the three run while DuckDB used the least amount of memory usage among the three optimization techniques. Meanwhile, Pandas performed slower and had lower throughput compared to Polars and DuckDB, especially when dealing with larget datasets. Even Pandas is easier to use and understand, the result showed that Polars abd DuckDB are more suitable for enhancing overall processing performance and efficiency in this project.

## 7.0 Challenges and Limitations

One of the main challenges encountered during this project was the data collection phase. At first, the JobStreet scraping procedure worked smoothly, and we were able to collect a significant amount of data with no serious concerns. However, the website began to identify the activity as bot-like behavior after the scraper was ran constantly for a longer amount of time in order to obtain the necessary dataset size. This resulted in HTTP 403 errors, which immediately banned access to specific pages. As a result, the scraping process got unstable and we were unable to keep it operating as intended. To prevent being blocked again, the process had to stop several times, the code needed to be modified and also reduce the scraping speed.

Another issue was an inconsistency in the data obtained. Some job postings had complete information such as the job title, company name, and location, but many others lacked crucial sections such as salary, company details or job description. Additionally, some variables were recorded as "N/A" or empty. As a result, the dataset was initially quite messy and not entirely prepared for analysis. As a result, significant time was spent cleaning data and dealing with missing values before the dataset could be used appropriately for performance evaluation and comparison.

## 8.0 Conclusion and Future Work

This project demonstrated the design, implementation and optimization process of a large-scale data processing pipeline to real-world job listing from JobStreet. The web crawler was built using Requests library to send http requests to JobStreet and BeautifulSoup to parse the returned HTML that contains structured fields from job card elements. The crawler incorporated randomised request delays, a three-attempt retry mechanism, and periodic cooldown intervals to ensure ethical and fault-tolerant data collection The pipeline successfully extracted 122,767 job listings, which was then reduced to 113,007 unique records after removing duplicates. The data cleaning process includes column standardization, duplicate removal, text field normalization, salary parsing, date parsing and URL validation. Three optimization libraries were used for the optimization comparison which are Pandas, Polars, and DuckDB. The three libraries were benchmarked against metrics such as processing time, CPU utilization, memory usage and throughput. The results showed that Polars was the best overall for fastest processing time and highest throughput with DuckDB having lowest memory consumption. Both libraries outperformed Pandas

which confirms the need for high-performance computing options for large-scale data processing.

For future work and improvements, the suggestions are listed as follows:

1. **Real-time Scraping** - Currently, the system performs a full re-crawl on every execution and collects all listings regardless if it has been previously recorded. A future version could include incremental web scraping logic and fetch only new or updated listings each run.

2. **Add Predictive Model** - The salary column has more than 50% records that has a non-disclosure rate. This means that less than half contributes to compensation and salary based analysis. A salary prediction model can be implemented by training the 41.13% of records with disclosed figures.

3. **Distributed Computing** - All three optimization libraries were evaluated on a single Google Colab instance which can have variability every run. Using distributed frameworks such as Dask or Apache Spark to evaluate how a dataset size scales beyond all records.

# Appendices

```
def parse_salary(val):
    """
    Parse salary strings covering: MYR (RM/MYR), SGD, IDR (Rp), PHP (₱),
    THB (฿), USD, HKD, AUD — plus shorthand like '5k', 'p.m.', 'p.a.'
    Returns: salary_min, salary_max, salary_currency, salary_period
    """
    if pd.isna(val):
        return pd.Series([np.nan, np.nan, np.nan, np.nan])

    raw = str(val).strip()
    low = raw.lower()

    # Freetext / non-numeric → NaN
    FREETEXT = {
        'tbd', 'ud', 'to be discussed', 'monthly salary', 'basic salary',
        'monthly', 'competitive salary', 'attractive salary package',
        'attractive salary', 'competitive pay', 'world class benefits',
        'basic + medical', 'basic + allowance + commission', '$0 - $0'
    }
    if low in FREETEXT or not re.search(r'\d', raw):
        return pd.Series([np.nan, np.nan, np.nan, np.nan])

    # ── Currency detection ──────────────────────────────────────────
    if   'IDR' in raw or re.search(r'\bRp\b', raw): currency = 'IDR'
    elif 'PHP' in raw or '₱' in raw:                currency = 'PHP'
    elif 'THB' in raw or '฿' in raw:                currency = 'THB'
    elif 'SGD' in raw:                               currency = 'SGD'
    elif 'HKD' in raw or 'HK$' in raw:              currency = 'HKD'
    elif 'AUD' in raw or 'A$' in raw:               currency = 'AUD'
    elif 'USD' in raw:                               currency = 'USD'
    elif re.search(r'\bRM\b|\bMYR\b', raw):         currency = 'MYR'
    else:                                            currency = 'Unknown'

    # ── Period detection ────────────────────────────────────────────
    if   re.search(r'p\.a\.|per year|yearly',  low): period = 'yearly'
    elif re.search(r'p\.h\.|per hour|hourly',  low): period = 'hourly'
    elif re.search(r'p\.m\.|per month|monthly',low): period = 'monthly'
    else:                                            period = 'unknown'
```

```
    # ── Number extraction ───────────────────────────────────────────
    if currency == 'IDR':
        # IDR uses dots as thousand-separators: Rp 10.000.000 → 10000000
        nums_raw = re.findall(r'\d[\d.]*', raw)
        nums = []
        for n in nums_raw:
            try: nums.append(float(n.replace('.', '')))
            except: pass
    else:
        # Expand shorthand: 10k → 10000, 1.5k → 1500
        cleaned = re.sub(
            r'(\d+(?:\.\d+)?)\s*k',
            lambda m: str(float(m.group(1)) * 1000),
            raw, flags=re.IGNORECASE
        )
        nums_raw = re.findall(r'\d[\d,]*(?:\.\d+)?', cleaned)
        nums = []
        for n in nums_raw:
            try: nums.append(float(n.replace(',', '')))
            except: pass

    nums = [n for n in nums if n > 0]  # drop zeros

    sal_min = nums[0] if len(nums) >= 1 else np.nan
    sal_max = nums[1] if len(nums) >= 2 else sal_min

    # Swap if inverted
    if pd.notna(sal_min) and pd.notna(sal_max) and sal_min > sal_max:
        sal_min, sal_max = sal_max, sal_min

    return pd.Series([sal_min, sal_max, currency, period])
```

```python
df[['salary_min', 'salary_max', 'salary_currency', 'salary_period']] = df['salary'].apply(parse_salary)

print('=== Currency breakdown ===')
print(df['salary_currency'].value_counts())
print()
print('=== Period breakdown ===')
print(df['salary_period'].value_counts())
print()
print(f"Rows with parseable salary : {df['salary_min'].notna().sum():,}")
print(f"Rows with no salary info   : {df['salary_min'].isna().sum():,}")
print()
# Spot-check one row per currency
for cur in ['MYR', 'SGD', 'IDR', 'PHP', 'THB', 'USD']:
    sample = df[df['salary_currency'] == cur][['salary', 'salary_min', 'salary_max', 'salary_period']].head(1)
    if not sample.empty:
        print(f'[{cur}]', sample.to_string(index=False))
```

```
=== Currency breakdown ===
salary_currency
MYR        13383
PHP         9898
SGD         9403
IDR         9084
THB         6295
USD          343
Unknown       88
AUD           21
HKD            3
Name: count, dtype: int64

=== Period breakdown ===
salary_period
monthly    46971
hourly       652
unknown      609
yearly       286
Name: count, dtype: int64

Rows with parseable salary : 48,456
Rows with no salary info   : 64,551

[MYR]                        salary  salary_min  salary_max salary_period
RM 6,800 - RM 10,000 per month      6800.0     10000.0       monthly
[SGD]                         salary  salary_min  salary_max salary_period
$2,800 - $3,500 per month (SGD)      2800.0      3500.0       monthly
[IDR]                                      salary  salary_min  salary_max salary_period
Rp 10.000.000 - Rp 12.000.000 per month (IDR)  10000000.0  12000000.0       monthly
[PHP]                           salary  salary_min  salary_max salary_period
₱15,000 - ₱16,500 per month (PHP)     15000.0     16500.0       monthly
[THB]                           salary  salary_min  salary_max salary_period
฿25,000 - ฿35,000 per month (THB)     25000.0     35000.0       monthly
[USD]                         salary  salary_min  salary_max salary_period
$700 - $900 per month (USD)       700.0       900.0       monthly
```

Appendix A: Parse Salary

Appendix B : Jobstreet Crawling Process